

Inventory & Purchase Management Automation

Technical specification for Airtable + n8n

Goal

I want to build a structured inventory and purchase-order management system using **Airtable + n8n**.

The goal is to know at any time the current inventory level of every product in each of three branches: **Beitar, Beit Shemesh, and Jerusalem**. Inventory should be updated automatically based on sales, stock receipts, and transfers between branches.

Each product will have a minimum desired stock level for each branch. The system should check inventory daily and send me an **email report** showing shortages, so I can know what needs to be reordered.

Airtable Structure - 4 Main Tables

1. Products - Products & Current Inventory

Each row represents one product/shade combination.

- SKU - unique product identifier
- Brand
- Product Name
- Shade
- Sale Price
- Cost Price
- Beitar Stock - current inventory
- Beit Shemesh Stock - current inventory
- Jerusalem Stock - current inventory
- Beitar Minimum - desired minimum stock
- Beit Shemesh Minimum - desired minimum stock
- Jerusalem Minimum - desired minimum stock
- Total Stock - total inventory across all branches
- Shortage - shortage compared with the defined minimum

The current stock values should be updated based on inventory transactions, rather than by manually changing stock numbers.

2. Inventory Transactions - Full Inventory Movement History

This table stores every action that changes inventory.

- Date & Time
- Product - linked to the Products table
- SKU
- Branch - Beitar / Beit Shemesh / Jerusalem
- Quantity Change
- Transaction Type - Sale / Stock Received / Transfer / Manual Correction

- Source - POS / AI Message / Purchase Order / Manual
- Reference - order number, file reference, or action ID
- Notes

Examples: sale of 2 units = **-2**; receipt of 10 units = **+10**. A transfer of 5 units from Beitar to Beit Shemesh should create two transactions: **-5 in Beitar** and **+5 in Beit Shemesh**.

Every inventory change must create a transaction so there is a complete audit trail and any discrepancy can be investigated.

3. Purchase Orders - Supplier Orders

- Order Number
- Supplier
- Order Date
- Product
- SKU
- Quantity Ordered
- Cost Price
- Total Cost
- Destination Branch
- Status - Draft / Ordered / Received
- Quantity Received
- Received Date

When stock is received and the order is marked as **Received**, n8n should automatically create a positive inventory transaction and update the stock of the relevant branch.

4. AI Sales Inbox - External Sales

This table is for sales that do not go through the POS. I will send free-text messages describing the sales, and AI should identify the relevant products.

- Date & Time
- Original Message
- Branch
- AI Parsed Result
- Matched Products
- Status - Processed / Needs Approval / Error
- Notes

n8n sends the message to the AI -> the AI identifies product, shade, quantity and branch -> n8n matches the result against the Products table -> if the match is certain, inventory transactions are created and stock is reduced.

If there is no reliable match, **do not change inventory automatically**. Mark the record as **Needs Approval** and send me an email alert.

How Sales Update Inventory

1. POS Sales

I will provide a file containing the sales from the POS. n8n should read the file, identify each product by SKU, identify the quantity and branch, create Inventory Transactions, and update the inventory.

If an SKU in the sales file does not exist in the Products table, the system should not silently ignore it. It should send me an email alert.

2. External Sales Using AI

I will send a free-text sales message, for example: "Sold in Jerusalem: 2 STATIC Rebecca, 1 Chloe, and 1 Inglot powder shade 12."

The AI should identify: **product + brand + shade + quantity + branch**. n8n should then match the result to existing Products records and create the required inventory transactions. If the identification is uncertain, inventory must not be updated automatically.

Minimum Stock & Daily Shortage Report

Each product should have a separate minimum stock level for each branch. Once per day, n8n should compare current inventory with the defined minimum and calculate the shortage.

Example: **STATIC Rebecca | Beitar | Current Stock: 3 | Minimum: 8 | Shortage: 5**

Required n8n Workflows

- 1 **POS Sales Import** - Receive sales file -> match by SKU -> create Transactions -> update inventory.
- 2 **External Sales AI** - Receive sales message -> AI parsing -> match products -> create Transactions -> update inventory.
- 3 **Purchase Receiving** - Mark supplier stock as received -> create positive Transactions -> update inventory.
- 4 **Stock Transfer** - Transfer between branches -> negative transaction at source + positive transaction at destination.
- 5 **Daily Low Stock Report** - Compare inventory to minimum levels once per day and email the shortage report.

Email Updates & Alerts

All system updates and alerts should be sent to my email.

- Daily report of all products below minimum stock, grouped by branch.
- Alert when AI cannot identify a product with sufficient confidence.
- Alert when an SKU from the POS sales file is not found in Airtable.
- Alert if there is an error while importing a sales file.
- Summary when a supplier order is received and inventory is added.
- Alert for any automation failure that causes a sale not to be reflected in inventory.

Overall System Flow

POS Sales / External Sales Messages / Stock Receipts / Branch Transfers

-> n8n

-> Inventory Transactions in Airtable

-> Updated inventory for all 3 branches

-> Comparison against minimum stock levels

-> Daily shortage report by email

-> Preparation of new supplier orders

Airtable should be the central database containing products, inventory, transactions and purchase orders. **n8n should handle the automation logic, AI processing, inventory updates and email notifications.**

It is important that every inventory change is logged in Inventory Transactions so that we can always trace exactly which action caused each change and correct errors when necessary.